Day 165 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 24, Day 5 reference

Watch: Model Calibration - Estimated Calibration Error (ECE) Explained (DataMListic) | Probability Calibration Workshop - Lesson 2 (numeristical, Brian Lucena)

`day165_calibration.py`, segment by segment, in the order `main()` uses things. The parts (reliability, ece, log_loss, logit, platt_loss, platt_fit, platt_apply, isotonic_fit, isotonic_apply, cross_fit) are copied unchanged into `day165_exercise.py`; the Adult data and logistic regression (Day 161), trees and boosting (Day 109) and ROC AUC (Day 117) are imported from their exercise files. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: p = predicted probabilities · y = 0/1 outcomes · n, mp, my = rows, mean prediction and share positive per bin · w = (a, b), Platt's slope and intercept · half = True for half A of the validation rows

Segment 0 — Setup

Imports and constants

CODE

```
sys.path.insert(0, os.path.join(ROOT, "week-16-classical-ml-trees"))
sys.path.insert(0, os.path.join(ROOT, "week-17-classical-ml-kernels-ensembles"))
sys.path.insert(0, HERE)
with contextlib.redirect_stdout(io.StringIO()):
    import day109_exercise as d109
    import day117_exercise as d117
    import day161_exercise as d161
BINS = 10
EPS = 1e-6
NAMES = ("logistic", "balanced logistic", "boosting", "deep tree")
```

- `import day109_exercise as d109` — Day 109's trees and boosting, unchanged.
- `EPS = 1e-6` — keeps probabilities away from exactly 0 and 1, where a logarithm is infinite.

Segment 1 — The parts

reliability and ece: binning with np.bincount

MATH

$$ECE = \Sigma_b \, (n_b/n) \, |mean\ p_b - share\ y_b|$$

CODE

```
def reliability(y, p, bins=BINS):
    b = np.minimum((p * bins).astype(int), bins - 1)
    n = np.bincount(b, minlength=bins)
    mp = np.bincount(b, p, bins) / np.maximum(n, 1)
    my = np.bincount(b, y, bins) / np.maximum(n, 1)
    return n, np.where(n > 0, mp, np.nan), np.where(n > 0, my, np.nan)
def ece(y, p, bins=BINS):
    n, mp, my = reliability(y, p, bins); ok = n > 0
    return float(np.sum(n[ok] * np.abs(mp[ok] - my[ok])) / len(y))
```

- `np.minimum((p * bins).astype(int), bins - 1)` — the bin of each row: 0.37 × 10 = 3.7, cut to 3; p = 1 would give 10, so it is capped at 9.

- `np.bincount(b, p, bins)` — with weights: the sum of p in each bin, in one pass.
- `np.where(n > 0, mp, np.nan)` — empty bins are marked NaN instead of a misleading 0.
- `n[ok] * np.abs(mp[ok] - my[ok])` — each bin's gap, weighted by its rows.

log_loss and logit: guarding the logarithm

CODE

```
def log_loss(y, p):
    p = np.clip(p, EPS, 1 - EPS)
    return float(-np.mean(y * np.log(p) + (1 - y) * np.log(1 - p)))
def logit(p):
    p = np.clip(p, EPS, 1 - EPS)
    return np.log(p / (1 - p))
```

- `np.clip(p, EPS, 1 - EPS)` — values below EPS become EPS, above 1 – EPS become 1 – EPS.
- `np.log(p / (1 - p))` — the log-odds; with the clip, 0 and 1 map to about –13.8 and +13.8.

platt_loss and platt_fit: Newton's method with step halving

MATH

$$\textit{Newton step} = H^{-1} g; \quad g = A^T(q - y), \quad H = A^T \, \mathrm{diag}(q(1 - q)) \, A$$

CODE

```
def platt_loss(A, y, w):
    z = A @ w
    return float(np.mean(np.logaddexp(0, z) - y * z))
def platt_fit(p, y, iters=100, damped=True):
    A = np.c_[logit(p), np.ones(len(p))]; w = np.array([1.0, 0.0])
    for _ in range(iters):
        q = 1 / (1 + np.exp(-np.clip(A @ w, -500, 500)))
        g = A.T @ (q - y)
        H = (A * (q * (1 - q))[:, None]).T @ A + 1e-9 * np.eye(2)
        step = np.linalg.solve(H, g); t = 1.0
        if damped:
            while platt_loss(A, y, w - t * step) > platt_loss(A, y, w) and t > 1e-10:
                t /= 2
        w = w - t * step
        if np.abs(t * step).max() < 1e-10:
            break
    return w
```

- `np.logaddexp(0, z)` — $\log(e^0 + e^z) = \log(1 + e^z)$, computed without overflow for large z.
- `np.c_[logit(p), np.ones(len(p))]` — two columns side by side: the log-odds and a column of ones for the intercept.
- `np.linalg.solve(H, g)` — solves H · step = g; cheaper and more accurate than inverting H.
- `while platt_loss(...) > platt_loss(...) and t > 1e-10:` — halve the step until it lowers the loss; this is what stopped the first draft running away.
- `damped=True` — a keyword argument with a default: the lesson calls damped=False once, to show the first draft.

isotonic_fit and isotonic_apply: a stack of blocks

CODE

```
def isotonic_fit(p, y):
    xs, inv = np.unique(p, return_inverse=True)
    wt = np.bincount(inv).astype(float); v = np.bincount(inv, y) / wt
    blocks = []
    for i in range(len(xs)):
        blocks.append([v[i], wt[i], i])
        while len(blocks) > 1 and blocks[-2][0] >= blocks[-1][0]:
            v2, w2, i2 = blocks.pop(); v1, w1, _ = blocks.pop()
            blocks.append([(v1 * w1 + v2 * w2) / (w1 + w2), w1 + w2, i2])
    return np.array([xs[b[2]] for b in blocks]), np.array([b[0] for b in blocks])
def isotonic_apply(m, p):
    right, val = m
    return val[np.minimum(np.searchsorted(right, p, side="left"), len(val) - 1)]
```

- `np.unique(p, return_inverse=True)` — the distinct scores, and for each row the position of its score among them.
- `blocks.append([...])` — a list used as a stack: new blocks go on the end.
- `v2, w2, i2 = blocks.pop()` — pop removes and returns the last block; unpacking names its three parts.
- `while len(blocks) > 1 and blocks[-2][0] >= blocks[-1][0]:` — keep merging backwards while the order is broken.
- `np.searchsorted(right, p, side="left")` — for each score, the first block whose right edge is at or above it.

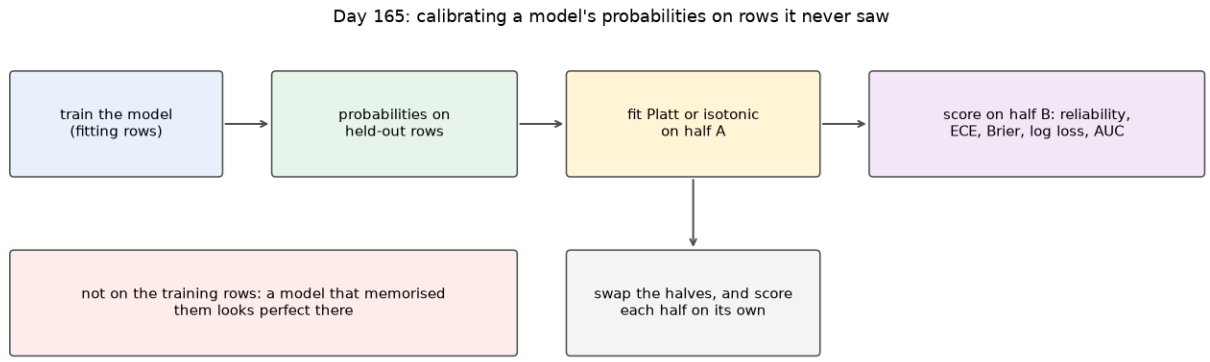
cross_fit: the swap in one loop

CODE

```
def cross_fit(p, y, half, fit, apply):
    out = np.zeros(len(p))
    for A, B in ((half, ~half), (~half, half)):
        out[B] = apply(fit(p[A], y[A]), p[B])
    return out
```

- `fit, apply` — functions passed as arguments: the same loop serves Platt and isotonic.
- `for A, B in ((half, ~half), (~half, half)):` — a tuple of pairs: fit on A, write into B, then the other way round.

SYSTEM DIAGRAM



Calibrating on rows the model never saw.

Segment 2 — The experiments

train_models: a balanced sample

CODE

```
rng = np.random.RandomState(0); pos = fi[y[fi] == 1]
bal = np.r_[pos, rng.choice(fi[y[fi] == 0], len(pos), replace=False)]
w, _ = d161.fit_logistic(Xo[bal], y[bal]); P["balanced logistic"] = d161.predict_logistic(w, Xo)
```

- `fi[y[fi] == 1]` — the fitting rows' indices where the target is 1.
- `rng.choice(..., len(pos), replace=False)` — an equal number of the other rows, each at most once.
- `np.r_[pos, ...]` — joins the two index arrays.

demo_measure: the chance floor

CODE

```
sim = [ece(rng.rand(len(p)) < p, p) for _ in range(200)]
```

- `rng.rand(len(p)) < p` — a new outcome for every row, positive with exactly probability p: a world where the model is perfectly calibrated.
- `for _ in range(200)` — _ is a name for a loop variable that is never used.

demo_fix: scoring each half on its own

CODE

```
ea, eb = ece(yv[half], q[half]), ece(yv[~half], q[~half])
fl = np.mean([ece(rng.rand(h.sum()) < q[h], q[h]) for h in (half, ~half) for _ in range(100)])
E[k][c] = ((ea + eb) / 2, fl)
```

- `q[half], q[~half]` — boolean indexing picks one half; ~ flips the mask to the other.
- `for h in (half, ~half) for _ in range(100)` — two for clauses in one comprehension: 100 simulations for each half.

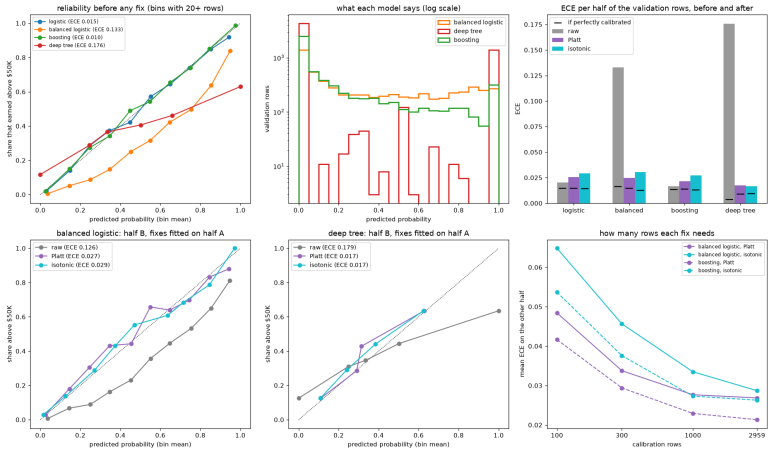
demo_size: many calibration sets in a generator

CODE

```
e = [ece(yb, a(f(pa[idx], ya[idx]), pb)) for idx in
     (np.random.RandomState(100 + s).choice(len(pa), n, replace=False) for s in range(50 if n <
         len(pa) else 1))]
```

- `(np.random.RandomState(100 + s).choice(...) for s in ...)` — round brackets make a generator: each index set is drawn only when the list needs it.
- `range(50 if n < len(pa) else 1)` — a conditional expression: 50 draws, or one when n is the whole half.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `np.bincount(b, weights, n)` — per-bin sums in one pass.
- `np.logaddexp(0, z)` — log(1 + e^z) without overflow.
- `np.linalg.solve(H, g)` — solve a linear system.
- `np.unique(..., return_inverse=True)` — distinct values and each row's position.
- `list.pop()` — remove and return the last item.
- `np.searchsorted(edges, x)` — where each x would go in sorted edges.

End of Day 165 syntax reference. For the experiments and what they showed, see the Day 165 complete guide.